

Create DataFrames

Keman Xiang

Background for this activity

Objective: create data frames with the `data.frame()` function to summarize and organize data in R.

A data frame is a collection of columns containing data, similar to a spreadsheet or SQL table. Data frames are one of the basic tools we will use to work with data in R. And we can create data frames from different data sources. This notebook is focused on creating and using data frames in R.

There are three common sources for data:

- A package with data that can be accessed by loading that package
- An external file like a spreadsheet or CSV that can be imported into R
- Data that has been generated from scratch using R code

Step 1: Load packages

Install `tidyverse`.

```
#install.packages("tidyverse")
```

```
library(tidyverse)
```

```
## — Attaching core tidyverse packages — tidyverse 2.0.0 —
## ✓ dplyr      1.1.4      ✓ readr      2.1.4
## ✓ forcats    1.0.0      ✓ stringr    1.5.1
## ✓ ggplot2    3.4.4      ✓ tibble     3.2.1
## ✓ lubridate  1.9.3      ✓ tidyr      1.3.0
## ✓ purrr      1.0.2
## — Conflicts — tidyverse_conflicts() —
## ✖ dplyr::filter() masks stats::filter()
## ✖ dplyr::lag()     masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

Step 2: Create data frame

Sometimes we will need to generate a data frame directly in R. There are a number of ways to do this; one of the most common is to create individual vectors of data and then combine them into a data frame using the `data.frame()` function.

First, create a vector of names:

```
names <- c("Peter", "Jennifer", "Julie", "Alex")
```

Then create a vector of ages:

```
age <- c(15, 19, 21, 25)
```

With these two vectors, we can create a new data frame called `people` :

```
people <- data.frame(names, age)
```

Step 3: inspect the data frame

Now that we have this data frame, we can use some different functions to inspect it.

One common function we can use to preview the data is the `head()` function, which returns the columns and the first several rows of data.

```
head(people)
```

```
##      names age
## 1   Peter  15
## 2 Jennifer  19
## 3   Julie  21
## 4    Alex  25
```

In addition to `head()`, there are a number of other useful functions to summarize or preview the data. For example, the `str()` and `glimpse()` functions will both provide summaries of each column in our data arranged horizontally.

```
str(people)
```

```
## 'data.frame':   4 obs. of  2 variables:
## $ names: chr  "Peter" "Jennifer" "Julie" "Alex"
## $ age : num  15 19 21 25
```

```
glimpse(people)
```

```
## Rows: 4
## Columns: 2
## $ names <chr> "Peter", "Jennifer", "Julie", "Alex"
## $ age <dbl> 15, 19, 21, 25
```

We can also use `colnames()` to get a list the column names in our data set.

```
colnames(people)
```

```
## [1] "names" "age"
```

Now that we have a data frame, we can work with it using all of the tools in `R`. For example, we could use `mutate()` if we wanted to create a new variable that would capture each person's age in twenty years.

```
mutate(people, age_in_20 = age + 20)
```

```
##      names age age_in_20
## 1   Peter  15         35
## 2 Jennifer  19         39
## 3   Julie  21         41
## 4    Alex  25         45
```

Step 4: Create Another DataFrame

First, create a vector of any five different fruits.

```
fruit <- c("Lemon", "Blueberry", "Grapefruit", "Mango", "Strawberry")
```

Now, create a new vector with a number representing our own personal rank for each fruit. Give a 1 to the fruit we like the most, and a 5 to the fruit we like the least. Remember, the scores need to be in the same order as the fruit above.

```
rank <- c(4, 2, 5, 3, 1)
```

Finally, combine the two vectors into a data frame. We can call it `fruit_ranks`.

```
fruit_ranks <- data.frame(fruit, rank)
```

It will create a data frame with our fruits and rankings.

Activity Wrap Up

In this notebook, we created data frames, viewed them with summary functions like `head()` and `glimpse()`, and then made changes with the `mutate()` function.